

# PROGRAMMING II

## COLLECTIONS

Michele Pasqua, PhD



UNIVERSITÀ  
di **VERONA**  
Dipartimento  
di **INFORMATICA**

A.Y. 2025/2026

# JAVA COLLECTIONS



## COLLECTION FRAMEWORK

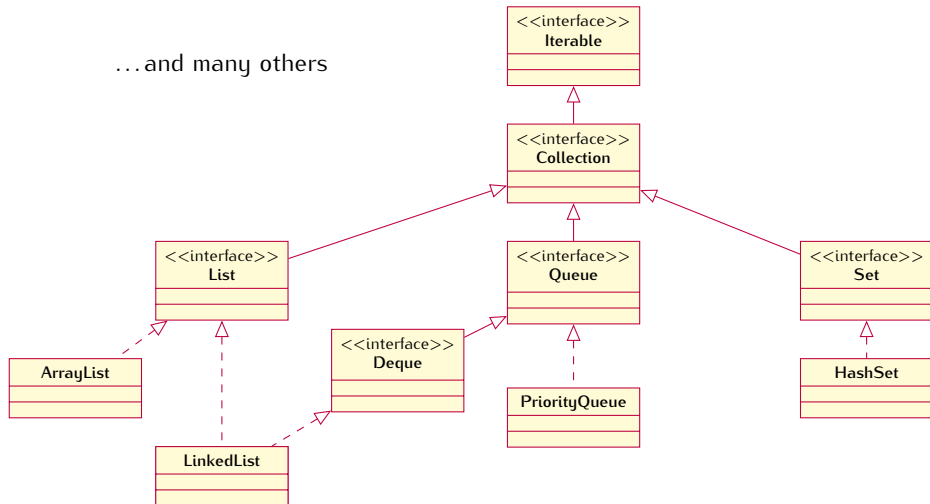
Set of classes and interfaces implementing **collections** of elements

- ▶ Sets, lists, queues, ...
- ▶ Different in how elements are grouped or accessed
- ▶ Defined in the `java.util` package
- ▶ All implementing the interface `IterableIterable<E>`

Since Java 5, the framework has been redefined by using generics

# COLLECTION HIERARCHY

...and many others



## DIFFERENT IMPLEMENTATIONS

Each collection has its own peculiarities

- ▶ Some collections are ordered and others are not
- ▶ Some collections have duplicates and others have not
- ▶ Some collections expose more functionalities and other less

Implementations can have different performance

|                      |                        | ArrayList | LinkedList |
|----------------------|------------------------|-----------|------------|
| Get an element       | <code>get(n)</code>    | constant  | linear     |
| Add element at first | <code>add(0, e)</code> | linear    | constant   |

```
public interface Collection<E> extends Iterable<E> {
    int size();
    int hashCode();
    boolean equals(Object obj); // equals is not generic ...
    boolean isEmpty();
    boolean contains(Object o);
    boolean containsAll(Collection<?> c);
    boolean add(E e);
    boolean addAll(Collection<? extends E>);
    boolean remove(Object o);
    boolean removeAll(Collection<?> c);
    void clear();
    Object[] toArray();
    Iterator<E> iterator(); // from Iterable<E>
}
```

## SWITCH IMPLEMENTATIONS

From lists to sets, to arrays, ...

*// or to differently implemented lists*

```
Collection<Person> list = new LinkedList<>();           1
list.add(new Person("Joe"));                             2
list.add(new Person("Sam"));                             3
                                                         4

Collection<Person> set = new HashSet<>(); // or new HashSet<>(list); 5
set.addAll(list);                                         6
                                                         7

Object[] arrayO = set.toArray();                          8
Person[] arrayP = set.toArray(new Person[0]); // ugly but useful 9
```

## LIST INTERFACE

Lists can have duplicates, and insertion order is preserved

- ▶ Elements can be accessed or inserted at a given position

Methods specific of the `List<E>` interface

- ▶ Get element at position: `E get(int i);`
- ▶ Replace element at position: `E set(int i, E e);`
- ▶ Add element at position: `void set(int i, E e);`
- ▶ Remove element at position: `E remove(int i);`
- ▶ Position of element: `int indexOf(E e);`
- ▶ Elements from 1 (inclusive) to r (exclusive): `List<E> subList(int l, int r);`

## LIST IMPLEMENTATIONS

`ArrayList<E> implements List<E>`

- ▶ Reserve space at instantiation: `ArrayList(int initialCapacity);`
- ▶ But the number of its elements is still zero

`LinkedList<E> implements List<E>, Queue<E>`

- ▶ Add as first: `void addFirst(E e);`
- ▶ Get last: `E addLast();`
- ▶ Remove first: `E removeFirst();`
- ▶ Push an element in the stack represented by the list: `void push(E e);`
- ▶ Pop an element from the stack represented by the list: `E pop();`

## SET INTERFACE

Sets cannot have duplicates, and insertion order is not preserved

- ▶ Elements cannot be accessed or inserted at a given position

The `Set<E>` interface contains no more methods than `Collection<E>`

- ▶ The elements in its `Iterator` are traversed with no particular order

## SET IMPLEMENTATION

One popular implementation is `HashSet<E>`

- ▶ Built on top of a key (hash) - value data structure
- ▶ Hash-based direct access to elements: constant time
- ▶ Hash computation: constant time

`HashSet<E>` implements `Set<E>`

- ▶ It internally maintains an `HashMap<E, Object>`
- ▶ Indeed, uniqueness of objects is primarily based on their hash code

# HASHING

The `Object` class contains the method `public int hashCode();`

- ▶ Returns an hash code of the object
- ▶ Used in hash-based data structures (e.g., `HashSet`)

It is correlated with the `public boolean equals(Object obj)` method

`o1.equals(o2)` **must imply** `o1.hashCode() == o2.hashCode()`

It is not strictly necessary the converse *// but improves performance*

`!o1.equals(o2)` **may imply** `o1.hashCode() != o2.hashCode()`

## HASHING (CONT'D)

Golden rule:

“if you change `equals()` you must also change `hashCode()`, and vice versa”

Moreover

- ▶ Equal objects must be associated with the same hash code
- ▶ Different hash codes must be associated with not equal objects

## CUSTOM HASH CODE

```
class Student extends Person { 1
    int matricola; 2
    ... 3
    @Override 4
    public int hashCode() { 5
        return name.hashCode() * matricola; 6
    } 7
    // the equals() method must be consistent -- 8
    // it must check name and matricola equality 9
} 10
```

From Java 7, you can use the method provided by the `java.util.Objects` class

`Objects.hash(name, matricola)`

## ORDERED SET IMPLEMENTATION

Another implementation is `TreeSet<E>`

- ▶ Built on top of a red-black tree data structure     *// self-balancing binary tree*
- ▶ Elements are sorted, but no duplicates are allowed
- ▶ Computationally expensive

Additional methods provided

- ▶ Get the first element of the set: `E first();`
- ▶ Get the last element of the set: `E last();`
- ▶ Elements between `from` (inclusive) and `to` (exclusive):  
`TreeSet<E> subSet(E from, E to);`

## ORDERED SET IMPLEMENTATION (CONT'D)

Difference in performance

- ▶ HashSet has constant add, remove, search time
- ▶ TreeSet has logarithmic add, remove, search time

Sorting and traversal are performed according

- ▶ `TreeSet()`: ordering from `Comparable<T>` elements
- ▶ `TreeSet(Comparator<E> comp)`: ordering from the given `Comparator<T>`

# ASSOCIATIVE CONTAINERS

A **map** is a set of associations between keys and values

(**key**) PassportId  $\Rightarrow$  Person (**value**)

Keys must be unique, and only one value can be associated to each key

- ▶ In Java, keys and values must be objects
- ▶ Implementation of the `java.util.Map` interface
- ▶ Even if part of the Collection Framework, it is not a subclass of `Collection`

# MAPS

```
public interface Map<K, V>
```

Methods provided by the interface

- ▶ Insert an association: `V put(K key, V values)`
- ▶ Get the value associated with a key: `V get(Object key)`
- ▶ Remove the association having a specific key: `V remove(Object key)`
- ▶ Check if the map contains a key: `boolean containsKey(Object key)`
- ▶ Check if the map contains a value: `boolean containsValue(Object value)`
- ▶ Returns the set of keys in the map: `Set<K> keySet()`
- ▶ Returns the collection of values in the map: `Collection<V> values()`

# MAP IMPLEMENTATIONS

The most common implementation is HashMap

*// put and get in constant time*

```
Map<String, Person> people = new HashMap<>();           1
                                                         2
people.put("J4DF67", new Person("Sam"));                3
people.put("A3GH41", new Person("Joe"));                4
                                                         5
System.out.println(people.size()); // the output is '2' 6
                                                         7
Person joe = people.get("A3GH41");                       8
if (bob == null) System.out.println("Not found");        9
                                                         10
if (!people.containsKey("A3GH41")) System.out.println("Not found"); 11
                                                         12
System.out.println(people.get("J4DF67")); // the output is 'Sam' 13
```

# WORKING WITH COLLECTIONS



# ITERATORS

A common task is to iterate on the elements of a collection

- ▶ Each collection is a subclass of `Iterable<E>`
- ▶ Hence each collection returns an `Iterator<E>`
- ▶ An `Iterator<E>` keeps track of the last element visited and moves on

The `Iterator<E>` class provides three methods

- ▶ Check if another element of the iteration is available: `boolean hasNext();`
- ▶ Returns the next element and moves on: `E next();`
- ▶ Remove the current element: `void remove();`

# ITERATORS (CONT'D)

Print all elements of a list

```
List<Integer> list = new ArrayList<>();           1
list.add(2);                                       2
list.add(3);                                       3

Iterator<Integer> it = list.iterator(); // using an iterator 4
while (it.hasNext()) {                             5
    System.out.println(it.next());                 6
}                                                    7
                                                    8
for (Integer i : list) { // using the enhanced for 9
    System.out.println(i);                         10
}                                                    11
                                                    12
```

Why should we use an iterator?

## ITERATE AND MODIFY

It is not safe to iterate a collection you are modifying at the same time

- ▶ Unless you use the `Iterator<E>` own methods

```

1 int count = 0;
2 for (Integer i : list) {
3     System.out.println(i)
4     if (count == 2) list.remove(count); // runtime error --
5     count ++;                          // ConcurrentModificationException
6 }
7 count = 0;
8 for (Iterator<Integer> it = list.iterator(); it.hasNext(); ) {
9     System.out.println(it.next());
10    if (count == 2) it.remove(); // ok
11    count ++;
12 }

```

# SORTING

Sortable collections (e.g., lists) have the overloaded method `sort`

On `Comparable<T>` objects

- ▶ `static <T extends Comparable<? super T>> void sort(List<T> list)`
- ▶ Comparable at design-time

By using a `Comparator<T>` object

- ▶ `static <T> void sort(List<T> list, Comparator<? super T> cmp)`
- ▶ Comparable on-demand

## SORTING WITH COMPARABLE

```
class Person implements Comparable<Person> {  
    ...  
    @Override  
    public int compareTo(Person other) {  
        return this.name.compareTo(other.name);  
    }  
}
```

1  
2  
3  
4  
5  
6  
7  
8

To sort a list `List<Person> list = new LinkedList<>()`

- ▶ Just use `Collections.sort(list);`
- ▶ The method `compareTo()` is used to sort

## SORTING WITH COMPARABLE (CONT'D)

Why <? **super** T> instead of <T>?

// <Person **super** Student>

```
// static <T extends Comparable<? super T>> void sort(List<T> list)      1
class Person implements Comparable<Person> {                             2
    ...                                                                    3
    @Override                                                            4
    public int compareTo(Person other) {                                  5
        return this.name.compareTo(other.name);                        6
    }                                                                    7
}                                                                          8
```

We have that Student **extends** Person and we want to inherit Person ordering

- ▶ Here Student (implicitly) extends Comparable<Student>
- ▶ But Student does not implement Comparable<Person>

## SORTING WITH COMPARATOR

Suppose neither Student nor Person implement Comparable

```
// static <T> void sort(List<T> list, Comparator<? super T> cmp)      1
class StudentComparator implements Comparator<Student> {              2
    public int compare(Student s1, Student s2) {                      3
        return s2.matr - s1.matr; // sort in reverse order           4
    }                                                                    5
}                                                                        6
```

To sort a list `List<Student> list = new LinkedList<>()`

- ▶ Just use `Collections.sort(list, StudentComparator);`
- ▶ Dynamic sorting definition

## SORTING WITH COMPARATOR (CONT'D)

Suppose neither Student nor Person implement Comparable

```
Collections.sort(list, new Comparator<Student>() {           1
    public int compare(Student s1, Student s2) {           2
        return s2.matr - s1.matr;                          3
    }                                                         4
});                                                         5
                                                           6
Collections.sort(list, (s1, s2) -> { return s2.matr - s1.matr; }); 7
```

Of course you can use an anonymous class or a lambda

## ARRAY SORTING

Also arrays have the overloaded method `sort`

On generic objects, i.e., on `Object`

- ▶ `static void sort(Object[] arr)`
- ▶ Compared on natural ordering (`compareTo()` is used, if given)

By using a `Comparator<T>` object

- ▶ `static <T> void sort(T[] arr, Comparator<? super T> cmp)`
- ▶ More type-safe

Sort by using `Arrays.sort(array);`

Q + A

